

Slot 10 · Find and fix five leaks

File: `leakage_audit.py` (Project Milestone 2) · **Done when:** `uv run pytest tests/test_slot10.py` shows 11 passed · **Time:** 18 min

Today's idea

Leakage means the model sees information during training that it will **not** have when a real order comes in. The score looks amazing. In real life, the model is much worse.

A colleague sends you a notebook: "*R² 0.99, up from 0.53. Recommend we ship Monday.*" You are the reviewer. Do you sign it off?

Step 1 · Open the notebook

```
uv run jupyter lab notebooks/leak_hunt.ipynb
```

A browser tab opens. Click **Run** → **Run All Cells**. Wait about 15 seconds. Near the end you should see:

```
R2    0.9884    MAE    0.816 min
```

MAE under one minute for a food delivery ETA. Too good to be true?

Step 2 · Five fixes, one at a time

There are five empty cells at the bottom, one per leak. In each, fix one thing and run `evaluate(train, test, F)` again. **Write down R² and MAE after every fix.**

#	kind of leak	your R ²	your MAE
1	a column that is the answer		
2	a column computed from the answer		
3	test-set information inside a training feature		
4	the same order on both sides of the split		
5	a split that lets the model see the future		

For leaks 1 and 2, read the `FEATURES` list slowly. For each column ask: "**Would I know this at the moment the customer places the order?**"

Remove a column like this (the next fixes build on `F`):

```
F = [c for c in FEATURES if c != "some_column"]
model = evaluate(train, test, F)
```

Stuck? Code for fixes 3, 4 and 5 is on the back of this page. Try on your own first.

Step 3 · Where you should end up

Around **R² 0.53** and **MAE 5.5 minutes**. Nothing about the model changed — only the honesty of the test.

Think 1. Which fix made the biggest drop? Was it the column with an obvious name? 2. One fix barely changed the score (it even went up a little). Which one, and why? (Hint: could *this* model memorise single orders?)

Step 4 · Write the audit as code

Open `leakage_audit.py`. Write `audit(feature_names)`: return the names that could not be known when the order is placed. `POST_OUTCOME` (already imported) lists the obvious ones.

Tip Also catch **derived** columns. A column called `rest_mean_payout` is built from `courier_payout_inr`, so it leaks too — but it is not in `POST_OUTCOME`. Check for parts of names like `"payout"`, `"actual_"`, `"delivered"`.

```
uv run python leakage_audit.py          # audits your features.py FEATURES list
uv run pytest tests/test_slot10.py
```

If something goes wrong

what you see	what to do
No module named <code>eta</code>	Run the first cell (it moves to the repo folder), then the others.
R^2 did not change after a fix	You changed <code>F</code> but did not re-run <code>evaluate</code> .
kernel died / very slow	Kernel → Restart Kernel and Run All Cells. Close extra browser tabs.
R^2 went up after a fix	That is fix 4. It is expected. Think about why.

Help: code for fixes 3, 4 and 5

Fix 3 — history from train only. The `rest_hour_mean` column was computed on **all** rows, including test rows. Recompute it from `train` only and join it onto both:

```
def add_history(tr, te):
    tr = tr.drop("rest_hour_mean", strict=False)
    te = te.drop("rest_hour_mean", strict=False)
    rh = tr.groupby(["restaurant_id", "hour"]).agg(
        pl.col("actual_minutes").mean().alias("rest_hour_mean"))
    g = tr["actual_minutes"].mean()
    join = lambda x: x.join(rh, on=["restaurant_id", "hour"], how="left") \
        .with_columns(pl.col("rest_hour_mean").fill_null(g))
    return join(tr), join(te)

train, test = add_history(train, test)
model = evaluate(train, test, F)
```

Fix 4 — drop the near-duplicate orders. `label()` removes them by default (the notebook called it with `drop_duplicates=False`):

```
d = label(raw)
shuffled = d.sample(fraction=1.0, shuffle=True, seed=SEED)
cut = int(0.8 * shuffled.height)
train, test = add_history(shuffled[:cut], shuffled[cut:])
model = evaluate(train, test, F)
```

Fix 5 — split by time. Train on the past, test on the last 45 days:

```
from eta.label import time_split
train, test = time_split(d)
train, test = add_history(train, test)
model = evaluate(train, test, F)
```

Reference ladder:

after fix	R ²	MAE (min)
none	0.9884	0.816
1	0.9817	1.164
2	0.6745	4.676
3	0.6420	4.929
4	0.6486	4.943
5	0.5343	5.490